EXPERIMENT 06

Text Generation using N-gram (Bigram) Language Model

AIM

To construct a Bigram language model from a text corpus and use it to probabilistically generate synthetic sentences in Python.

PROCEDURE

1. Tokenize the training corpus into words and normalize to lowercase.
2. Formulate consecutive bigrams (w_{i-1}, w_i) and compute transitional frequency distribution.
3. Construct a conditional frequency and probability mapping from each preceding word to candidate next words.
4. Define a sentence generation loop that samples next words sequentially given the current word.
5. Generate sample sentences starting from specified trigger seeds.

PROGRAM

```
import random
from collections import defaultdict, Counter

corpus = """natural language processing makes computers understand human language.
natural language processing enables machines to read and summarize text.
natural language understanding is a subfield of artificial intelligence.
language models generate coherent and fluent text for conversational agents.
artificial intelligence transforms how computers communicate with human users."""

words = corpus.lower().replace('.', ' <end>').split()

bigram_model = defaultdict(Counter)
for i in range(len(words) - 1):
    w1, w2 = words[i], words[i+1]
    bigram_model[w1][w2] += 1

def generate_text(seed, max_len=12):
    current = seed.lower()
    output = [current]

    for _ in range(max_len - 1):
        if current not in bigram_model:
            break
        next_candidates = bigram_model[current]
        words_list = list(next_candidates.keys())
        weights = list(next_candidates.values())

        next_word = random.choices(words_list, weights=weights)[0]
        if next_word == '<end>':
```

```
            break
        output.append(next_word)
        current = next_word

    return ' '.join(output)

random.seed(42)
print("--- Bigram Model Generated Sentences ---")
for seed in ["natural", "artificial", "language", "computers"]:
    generated = generate_text(seed, max_len=10)
    print(f"Seed: [{seed}] -> {generated}")
```

OUTPUT

```
> python3 experiment_06.py
--- Bigram Model Generated Sentences ---
Seed: [natural] -> natural language processing enables machines to read and summarize
text
Seed: [artificial] -> artificial intelligence transforms how computers understand human
language
Seed: [language] -> language models generate coherent and fluent text for conversational
agents
Seed: [computers] -> computers communicate with human users
[Process completed with exit code 0]
```

RESULT

Thus, the N-gram (Bigram) language model was successfully implemented in Python to model transition distributions and generate coherent text sequences.